Viva preparation

Prepared answers to the question list in the Development Plan (Section 8.2), then the one-page sheets (8.3). Each answer is short enough to say aloud and names the figure or number to point at. Derivations are in appendix_math.md; practise them on paper, not from this page.

How to use it: one of you asks, the other answers without notes and derives on paper where a question says *derive*. Swap. Then repeat with the other's modules (gate1.md Section 4 and gate2.md Section 4 list who explains what).

---

1. Prepared answers

The big picture

1. Why a CVAE instead of a plain MLP that maps a room to one layout? A room has many good layouts. A network trained to output one would learn their average, and the average of a sofa against the north wall and a sofa against the south wall is a sofa in the middle of the room. The CVAE's latent variable z selects *which* good layout to produce, so different samples give different layouts. We also need several candidates per request, because not every candidate is valid.

2. Your generator already makes 100% valid layouts, so why train a network? Say this plainly; it is the question most likely to be asked.

- For an ordinary request the generator is the better tool. Per valid layout it costs 2.4 ms against 21 ms for M2, and its layouts score 0.88 on quality against 0.66 for M2's (0.85 for the three M2 shows).
- The learned pipeline does beat the samplers that have no template: 69.8% raw valid against 57.8% for B2 and 10.1% for B1.
- We hoped pinned furniture would be the learned pipeline's case. E12 says no. With one item pinned, the generator with that item moved to its spot (G0-pin) gives 47% valid samples at quality 0.80; M2 gives 39% at 0.67, for six times the cost. The pin mechanism itself works: M2 brings the item to within 0.14 m of its pin before placing it, and 99% of requests get a valid layout.
- The project's purpose is the controlled study of the deep learning parts, with the generator as an honest reference: which loss, activation and optimizer work and why, whether repair by gradient descent in latent space works (it quadruples validity), and what a CNN can see in a layout image.

3. The rule score is exact, so what does the CNN evaluator add? Not correctness and not speed: the checker decides validity, and the rule check is as fast as the CNN. It adds three things. A controlled CNN study (E9a: 96.6% accuracy overall, but only 81% on near-miss overlaps). A learned ranker whose value we measured: it lifts M2's top 3 from 0.67 (random order) to 0.85, and ranking by the exact rule score would give 0.88. And a starting point for human preference ratings, which no rule score captures.

The CVAE

4. Derive the KL term. What is the reparameterization trick and why is it needed? *(derive)* For one dimension, $\mathrm{KL}(\mathcal{N}(\mu, \sigma^2) \,\|\, \mathcal{N}(0,1)) = \frac{1}{2}(\mu^2 + \sigma^2 - 1 - \log \sigma^2)$, from $\mathbb{E}[(z-\mu)^2] = \sigma^2$ and $\mathbb{E}[z^2] = \mu^2 + \sigma^2$; sum over dimensions (appendix 6.2). Sampling z directly has no gradient with respect to μ and σ. Writing $z = \mu + \sigma\epsilon$ with $\epsilon \sim \mathcal{N}(0, I)$ moves the randomness into ϵ, so $\partial z / \partial \mu = 1$ and the reconstruction gradient reaches the encoder.

5. What is posterior collapse? How did you detect it? The encoder outputs the prior for every input, the KL is 0, and the decoder ignores z: every sample for a room looks the same. We count *active units*, latent dimensions whose mean KL on validation data exceeds 0.01 nats. The frozen models have 9 or 10 of 16. We saw a real collapse once, on purpose: Sigmoid, six layers, no BatchNorm in E3b had 0 active units.

6. Why Sigmoid on positions? What happens near 0 and 1? Positions are fractions of the room in $[0, 1]$, the Sigmoid's range, so no output can leave the room by much. Near 0 and 1 its gradient vanishes, so we expected larger errors for items against walls. E7 found the opposite: 0.55 m near walls, 0.58 m elsewhere, because wall items are the most predictable. A linear head with clamping was no better.

7. Why Softmax with cross-entropy for rotation? Derive the gradient. Why logits? *(derive)* Facing is one of four classes, not a number: 270° is not "three times" 90°. With $p = \mathrm{softmax}(z)$ and $L = -\sum_i y_i \log p_i$, $\partial L / \partial z_k = p_k - y_k$ (appendix 2.2). `F.cross_entropy` takes the logits and computes log-softmax in one numerically stable step; applying a softmax first and then a log can give $\log 0$.

8. MSE vs MAE vs Huber: gradients and outliers. What did E2 show? MSE's gradient $2e$ grows with the error; MAE's is ± 1; Huber is quadratic within δ and linear beyond. E2: MAE reconstructs three times better than MSE (0.19 m against 0.58 m), Huber is worst (0.84 to 0.90 m). The reason is the loss *scale* against the fixed KL weight, not outliers: MAE's term is about ten times larger than MSE's at typical errors, so more information stays in z. With 4% outliers nothing much changed for any loss. MAE was then confirmed with three seeds on validation rooms and frozen.

9. What did E3 show about vanishing gradients and dying ReLU? Why turn BatchNorm off? Without BatchNorm at six layers: Sigmoid's gradient at the first layer is 10^{-10} and it collapses (loss 4.87); 29% of ReLU units are dead (loss doubles to 1.24); ELU and GELU are fine. BatchNorm hides both: deep ReLU then has no dead unit. The deployed model keeps BatchNorm, so the effects had to be provoked by removing it; the BatchNorm-on runs at depth 6 are the control.

10. What does BatchNorm do in training and at inference? Why `eval()` during latent optimization? In training it normalizes each unit with the *batch's* mean and variance and updates running averages; at inference it uses the running averages. During latent optimization the 64 candidates of one request form the batch. In training mode each candidate's output would depend on the other 63, and the running averages would be overwritten by one room's statistics. `eval()` makes the decoder a fixed function of z.

11. What does dropout do? What did E6 show about the train-validation gap? It zeroes random units in training so the network cannot rely on single units; at inference it is off. E6: the gap is 0.002 (0.569 training, 0.571 validation), so the CVAE does not overfit, and dropout 0.3 only hurts. The trap: the loss logged *during* training is higher than the validation loss because dropout is active then. Both splits must be scored in inference mode.

12. Compare SGD, momentum, RMSProp and Adam. Write the Adam update. What did E4 show? *(derive)* Momentum accumulates past gradients; RMSProp divides each parameter's step by a running size of its gradient; Adam does both with bias correction (appendix 5). E4: Adam at 10^{-3} reaches the target loss in 24 epochs and the best final loss; plain SGD at 0.01 needs 153. Adaptive methods tolerate a wrong learning rate far better than SGD.

23. Where is early stopping used, and why does it start after annealing? On the CVAE's validation loss, always computed at the final β, with patience 15. While β rises from 0 to 0.1 over the first 20 epochs, the training objective changes and the loss can climb; a stopper running then would keep a checkpoint trained almost without the KL term. A test proves it cannot fire before epoch 21.

Latent optimization

13. Why is latent optimization not cheating? It only moves candidates. After it, the same rule checker decides, unchanged, and it rejects 30% of the repaired candidates. The optimization target does not even contain reachability, so a repaired layout can still fail H4.

14. Why penetration depth and not intersection area as the loss? What happens to a contained box? *(derive)* If one box lies inside another along an axis, the overlap width equals the smaller box's width wherever it sits, so the area's gradient is zero: no direction to move. The penetration depth $\mathrm{ReLU}((w_i + w_j)/2 + \mu - |x_i - x_j|)$ has gradient $-\mathrm{sign}(x_i - x_j)$: it always pushes the centres apart. A test and the notebook show both facts.

15. Why an anchor to z_0 and not a $\|z\|^2$ prior? Shrinking $\|z\|$ pulls every candidate towards the same point, $z = 0$, and they would all become the same layout. The anchor keeps each one near its own start. E8 confirms it: the diversity ratio does not fall with more steps (0.91 at 0 steps, 0.95 at 200).

Data and representation

16. Why does the loss use a presence mask? A room has between two and six items. Absent slots have zero targets; without the mask the network would be trained to predict zeros for them and their error would dominate. Multiplying each item's loss by $m_k \in \{0, 1\}$ removes both its loss and its gradient.

17. Why divide coordinates by room size? The target becomes a fraction in $[0, 1]$ whatever the room, matching the Sigmoid, and "against the east wall" is always $u \approx 1$. That is part of why M2 holds up in rooms larger than any in training (E10).

18. Why canonicalize rotations? What would go wrong otherwise? A coffee table looks the same after a half turn. If the data stored either facing at random, identical layouts would have different targets and the rotation head would be trained towards 50/50 between them: pure noise in the loss. One canonical form per layout removes it. (Swapping nightstands is the same idea for the bedroom, which we did not build.)

22. Your data is synthetic and rejection sampling skews it. Consequences? What did you do? The model can only learn our rules' idea of a good layout, not a designer's. Rejection makes crowded arrangements rarer: 79% of attempts fail in the smallest rooms. We kept the room fixed for up to 20 attempts so that 99.3% of rooms still appear, logged the rejection rate by room area, calibrated a crowding limit ($f_{\max} = 0.38$) beyond which the pipeline refuses, and state the limitation.

25. Why does the checker treat every face of a side table alike? Its stored facing is arbitrary (canonical form always stores 0). If reachability checked only the "front", a side table with its stored front against a wall would be called unreachable although it can be reached from any other side, and valid layouts would fail H4 after canonicalization.

Evaluation

19. Precision vs recall for "valid": which error is worse? What is your near-miss F1? Calling an invalid layout valid (low precision) would be worse *if the CNN decided*, since a user would see a broken layout. It does not decide: the checker does, so either error only changes the ranking. Precision 0.967, recall 0.980. On near-miss overlaps the F1 (invalid as the positive class) is 0.82 and the accuracy 81%, the weakest of all types.

20. Show the matrix shapes through the decoder for a batch. *(derive)* Input $[z; c]$: $B \times 41$ (16 + 25). Hidden layers: $B \times 256$, twice (W_1 is 41×256, W_2 is 256×256). Position head: $B \times 12$, reshaped to $B \times 6 \times 2$. Rotation head: $B \times 24$, reshaped to $B \times 6 \times 4$.

21. What happens when the room is bigger than anything in training? Unseen combination vs out of range? Validity does not fall; it rises, because larger rooms are easier: M2 goes from 70% in distribution to 74% (interpolation), 84% (unseen combination) and 87% (out of range), while the generator's own acceptance rises from 75% to 96%. Against that reference M2 keeps 90 to 93% everywhere. Quality is where it costs: out of range the top 3 fall from 0.85 to 0.77, the generator's only from 0.89 to 0.86. Unseen combination: areas above 32 m^2, where each width and each depth also occurs in training, but never together. Out of range: widths of 7 to 8 m and depths of 6 to 7 m, values no training room has. The app warns for such rooms.

24. What would you do with more time or real data? Add a differentiable reachability term or the CNN as a surrogate in latent optimization, since reachability is M2's main remaining failure. Train on designer-made rooms (3D-FRONT) so the model learns taste rather than our rules. Replace the fixed six slots with a set-based model (a transformer) to allow any number of items, add windows and non-rectangular rooms, and tune the ranker on human ratings.

26. Can someone else reproduce your numbers? *(not in the plan's list, but asked of every project)* On the same machine, exactly, and we checked it. Every random stream is seeded and PyTorch runs in deterministic mode. On 9 October we ran every step again into an empty folder, reusing nothing: all 21 result tables came out equal outside their timing columns, all 64 trained models and the dataset were identical bit for bit, and Gate 2 chose the same configuration. Two things do not reproduce. Timings change with the laptop's power and heat, which is why we time the methods in turn and quote ratios. And across devices the same seed gives different random numbers (CPU against GPU), so on another machine we would expect close, not identical, results. Point at `reports/regeneration.md`.

---

2. One-page sheets

In the PDF (`python run.py pdf`, `reports/pdf/viva_prep.pdf`) the sheets start on a page of their own, so they can be printed alone.

2.1 Formulas to know by heart

BCE with Sigmoid	$\partial L/\partial z = p - y$
Softmax with cross-entropy	$\partial L/\partial z_k = p_k - y_k$
MSE, MAE gradients	$2e$, $\operatorname{sign}(e)$
One layer, $H = \operatorname{act}(XW + b)$	$\partial L/\partial W = X^\top \Delta$, $\partial L/\partial X = \Delta W^\top$, $\Delta = \partial L/\partial H \odot \operatorname{act}'(Z)$
KL to $\mathcal{N}(0, I)$	$\frac{1}{2}\sum_j(\mu_j^2 + \sigma_j^2 - 1 - \log\sigma_j^2)$
Reparameterization	$z = \mu + \sigma \odot \epsilon$
CVAE loss	masked reconstruction $+ \beta\cdot$ KL, β: 0 to 0.1 over 20 epochs
Adam	$\theta \leftarrow \theta - \eta\,\hat{m}/(\sqrt{\hat{v}} + \epsilon)$
Penetration depth	$\mathrm{ReLU}((w_i + w_j)/2 + \mu - \lvert x_i - x_j\rvert)$, the smaller of the two axes
Latent optimization	$10\sum \mathrm{pen}^2 + 10\sum \mathrm{out} + 10\sum \mathrm{pen}^2_{\mathrm{door}} + 0.05 \cdot \frac{1}{2}\lVert z - z_0\rVert^2$
Quality score	$0.30\,\mathrm{align} + 0.30\,\mathrm{relations} + 0.25\,\mathrm{circulation} + 0.15\,\mathrm{space}$
Door centre	$s = 0.65 + (L - 1.3)\,o$

2.2 Architecture

```
Encoder q(z | x, c)                          Decoder p(x | z, c)
[x ; c]              61                      [z ; c]              41
Linear 61 -> 256, BatchNorm, ReLU, Dropout   Linear 41 -> 256, BatchNorm, ReLU, Dropout
Linear 256 -> 256, BatchNorm, ReLU, Dropout  Linear 256 -> 256, BatchNorm, ReLU, Dropout
Linear 256 -> 16  mu                         Linear 256 -> 12, Sigmoid   positions (6 x 2)
Linear 256 -> 16  log variance               Linear 256 -> 24            rotation logits (6 x 4)
                                             177,732 parameters in all

CNN evaluator: 4 x 128 x 128 image -> [Conv 3x3, BatchNorm, ReLU, MaxPool] x 4 (16, 32, 64, 64 channels)
               -> 64 x 8 x 8 -> Linear 128, ReLU, Dropout -> valid logit, quality score     585,682 parameters

Pipeline: request checks -> variants within budget and f_max -> 64 samples of z -> latent optimization (150 steps)
          -> rule checker H1 to H4 -> CNN ranking -> top 3 at least 0.3 m apart
```

2.3 Results to quote

Frozen configuration (MAE), three seeds, 500 test rooms, 64 samples each.

Method	Raw valid	Quality (valid)	Top 3 shown	Diversity ratio	Cost per valid layout
B1 uniform	10.1%	0.29	0.30	0.98	18 ms
B2 statistical	57.8%	0.55	0.62	1.02	4.0 ms

Method	Raw valid	Quality (valid)	Top 3 shown	Diversity ratio	Cost per valid layout
G0 generator (reference)	74.7% per attempt	0.88	0.89	1.00	2.4 ms
M1 CVAE	17.3 ± 1.9%	0.66	0.78	0.91	10.5 ms
M2 CVAE + latent optimization	**69.8 ± 0.9%**	0.66	**0.85**	0.95	21 ms

Other numbers: latent optimization 17% → 63% (25 steps) → 69% (200 steps). Evaluator 96.6% accuracy, 81% on near-miss overlaps. Freeze: MAE 72.1% against 67.1% for the default on validation rooms; Tanh 64.0%. E10: M2 70% in distribution, 74%, 84% and 87% on the held-out sets (90 to 93% of G0's rate on each); top-3 quality 0.85 falling to 0.77 out of range. Pinned furniture (E12): valid samples B2 62%, G0-pin 47%, M2 39%; quality 0.57, 0.80, 0.67.

2.4 Assumptions (ours, not a standard)

- Door 0.9 m wide with a 0.9 m deep clearance zone, at least 0.65 m from a corner.
- A person needs 0.3 m of clearance on each side to pass; an item is reached from 0.35 m in front of its face.
- Two items may overlap by at most 0.005 m^2.
- Sofa to TV unit 1.5 to 3.5 m; coffee table 0.35 to 0.5 m in front of the sofa; furniture covers 15 to 40% of the floor.
- Score weights 0.30, 0.30, 0.25, 0.15.
- Furniture sizes and prices are placeholders.
- Rooms 3.5 to 7 m wide and 3 to 6 m deep; optional furniture is more likely in larger rooms.

2.5 Limitations to state before being asked

- Synthetic data: the model learns our rules' idea of a good layout, not a designer's.
- The rule-based generator is cheaper than the learned pipeline and its layouts score higher, with or without pinned furniture.
- Rectangular rooms, one door, six items, four facings; no windows.
- Reachability is not in the optimization target, and it is M2's main remaining failure.
- Three seeds describe the spread; they are not a significance test.
- The first shortlist of settings used test rooms for M1; the final choice used validation rooms only.
- Timings come from a laptop whose speed changes with its power state; they were measured with the methods taking turns, and only their ratios should be quoted.
- Not architectural or safety advice: fire exits and accessibility rules are not modelled.